



Navigation Assistance System for Differently-Abled Passengers

Complete Questions & Answers

Comprehensive Q&A covering project overview, YOLOv8 object detection, Flask web development, BFS graph navigation, OpenCV visualization, TTS audio guide, dataset preparation, and code-level viva questions.

[Python](#)[YOLOv8](#)[Flask](#)[OpenCV](#)[BFS Algorithm](#)[pyttsx3 / gTTS](#)[PyTorch](#)[PIL / Pillow](#)[Threading](#)[REST API](#)



Table of Contents

1.	Project Overview & Architecture	15 Qs
2.	YOLOv8 & Object Detection	20 Qs
3.	Flask Web Application & REST API	18 Qs
4.	BFS Navigation & Graph Routing	15 Qs
5.	OpenCV & Visualization	12 Qs
6.	Audio Guide & Text-to-Speech	10 Qs
7.	Dataset Preparation & Synthetic Data	15 Qs
8.	Training Pipeline	12 Qs
9.	Configuration & Settings	8 Qs
10.	Python Concepts Used	12 Qs
11.	Code-Level Viva Questions	20 Qs
12.	Real-World & Application Questions	10 Qs

1 Project Overview & Architecture

1 What is the Navigation Assistance System? Theory

The Navigation Assistance System is an **AI-powered application** designed to help differently-abled passengers navigate railway stations. It uses **YOLOv8 object detection** to detect passengers via camera feeds and provides **real-time audio and visual guidance** to help them reach their destination within the station.

The system identifies 6 classes of passengers: person, wheelchair user, blind person, person with crutches, elderly person, and person with luggage. It maps detected positions to predefined station zones and uses **BFS (Breadth-First Search)** to compute the shortest route to the destination.

2 What are the main technologies used in this project? Theory

Technology	Purpose
Python 3.8+	Core programming language
YOLOv8 (Ultralytics)	Real-time object detection
Flask	Web application framework
OpenCV (cv2)	Image/video processing & visualization
PyTorch	Deep learning backend for YOLO
pyttsx3 / gTTS	Text-to-Speech (offline / online)
PIL (Pillow)	Synthetic image generation
NumPy	Numerical array operations
Threading	Background tasks (training, audio, camera)
BFS Algorithm	Shortest path navigation

3

Explain the project directory structure.

Theory

```

navigation-assistance/
├─ app.py           # Flask web application (main entry)
├─ demo.py          # Standalone demo (pretrained model)
├─ train.py         # CLI training script
├─ detect.py        # CLI detection (image/video/live)
├─ download_dataset.py # Dataset download/generation
├─ prepare_dataset.py # Dataset organiser & verifier
├─ gen_synthetic.py  # Synthetic dataset generator
├─ config/
│   ├─ settings.py  # Zone layout, class colors, thresholds
│   └─ dataset.yaml # YOLO dataset configuration
├─ navigation/
│   ├─ navigator.py  # BFS graph routing (9 zones)
│   └─ audio_guide.py # TTS audio announcements
├─ utils/
│   ├─ detector.py   # YOLOv8 inference wrapper
│   └─ visualization.py # Bounding boxes, zone overlay, arrows
├─ templates/        # Jinja2 HTML templates
├─ static/            # CSS, JS, uploads
├─ models/            # Saved .pt model weights
└─ data/dataset/      # Training images & labels

```

4

What are the different modes of running this application?

Theory

The system can be run in **3 main ways**:

1. **Demo Mode** (`python demo.py`) — Instant demo using pretrained YOLOv8n. Supports live webcam, video, image, or synthetic animated demo (`--no-camera`).
2. **Web App** (`python app.py`) — Full Flask web UI at `http://127.0.0.1:5000` with pages for Dashboard, Demo, Train, Detect, and Navigation.
3. **CLI Detection** (`python detect.py`) — Command-line detection on images, videos, or live camera with optional audio guidance.

5

What are the 6 detection classes and why were they chosen?

Theory

ID	Class Name	Color (BGR)	Purpose
0	person	Blue (219,86,26)	General passenger
1	wheelchair_user	Orange (0,140,255)	Mobility-impaired passenger in wheelchair
2	blind_person	Green (0,200,100)	Visually impaired passenger
3	person_with_crutches	Purple (255,100,0)	Passenger with walking aids
4	elderly_person	Yellow (180,0,255)	Senior citizen passenger
5	person_with_luggage	Cyan (0,200,200)	Passenger carrying bags/luggage

These classes were chosen to cover the primary categories of differently-abled and special-needs passengers at railway stations who need navigation assistance.

6

What are the 9 station zones defined in the system?

Theory

The station layout is divided into **9 zones**, each defined as fractional coordinates (x1, y1, x2, y2) of the video frame:

1. **Entrance** (0.30, 0.00, 0.70, 0.20)
2. **Main Hall** (0.20, 0.20, 0.80, 0.60)
3. **Ticket Counter** (0.70, 0.00, 1.00, 0.40)
4. **Platform 1** (0.00, 0.40, 0.35, 1.00)
5. **Platform 2** (0.65, 0.40, 1.00, 1.00)
6. **Restroom** (0.00, 0.00, 0.30, 0.40)
7. **Elevator** (0.40, 0.55, 0.60, 0.80)
8. **Waiting Area** (0.20, 0.55, 0.80, 0.80)
9. **Exit** (0.30, 0.80, 0.70, 1.00)

These fractional coordinates are multiplied by frame width and height to map pixel positions to zones.

7

Explain the overall data flow of the system from camera input to audio output.

Theory

1. **Camera Capture:** OpenCV captures video frames from webcam or video file.
2. **Object Detection:** Each frame is passed to YOLOv8 model for inference → returns bounding boxes, class IDs, confidence scores.
3. **Result Formatting:** Raw YOLO results are converted to structured dictionaries with class name, confidence, bbox coordinates, and center point.
4. **Zone Mapping:** The center (cx, cy) of each detection is normalized to (fx, fy) and matched against predefined zone boundaries.
5. **Route Computation:** BFS algorithm computes shortest path from detected zone to destination zone.
6. **Visualization:** Annotated frame is drawn with bounding boxes, zone overlays, navigation arrows, and HUD.
7. **Audio Guide:** The first step instruction is spoken via pyttsx3 (offline) or gTTS (online fallback) with debounce to avoid repetition.
8. **Output:** Annotated frame displayed via OpenCV window or streamed as MJPEG to browser.

8

What is the role of `requirements.txt` ? List key dependencies.

Theory

`requirements.txt` lists all Python packages needed. Key dependencies:

- `flask ≥ 2.3.0` — Web framework
- `ultralytics ≥ 8.0.0` — YOLOv8 library
- `opencv-python ≥ 4.8.0` — Computer vision
- `torch ≥ 2.0.0` & `torchvision ≥ 0.15.0` — Deep learning backend
- `pyttsx3 ≥ 2.90` — Offline text-to-speech
- `gtts ≥ 2.3.0` — Google TTS (online fallback)
- `Pillow ≥ 9.0.0` — Image generation
- `roboflow ≥ 1.1.0` — Dataset download from Roboflow

Install with: `pip install -r requirements.txt`

9

How does the system handle the case when no model is loaded?

Code

The `Detector` class has an `is_loaded()` method that checks if `self.model is not None`.

- In **app.py** (live stream): If model is not loaded, the raw frame is displayed with a red text overlay: `"No model loaded – use /api/load_model"`.
- In **API endpoints** (detect/image, detect/video): Returns JSON error: `{'success': False, 'message': 'No model loaded. Please load a model first.'}`
- In **detector.py**: The `detect()` method returns `None` if model is not loaded, and `format_results()` returns an empty list for `None` results.

10

What design pattern does the project use for its modules?

Theory

The project uses a **modular architecture** with **separation of concerns**:

- **Detector** (utils/detector.py) — Encapsulates YOLO inference (Single Responsibility)
- **Visualizer** (utils/visualization.py) — Handles all drawing operations
- **Navigator** (navigation/navigator.py) — Route planning & zone mapping
- **AudioGuide** (navigation/audio_guide.py) — TTS with fallback pattern (Strategy Pattern)
- **Settings** (config/settings.py) — Centralized configuration (Configuration Pattern)
- **app.py** — Controller/orchestrator that ties everything together

The AudioGuide also uses a **fallback/chain** pattern: tries pyttsx3 → gTTS → console print.

11

What are the web UI pages available and what does each do?

Theory

Page	URL	Purpose
Dashboard	<code>/</code>	Load model, system overview, list saved models
Demo Mode	<code>/demo</code>	Instant demo, dataset download options
Train	<code>/train</code>	Upload dataset, configure & start model training
Detect	<code>/detect</code>	Image / Video / Live camera detection
Navigation	<code>/navigation</code>	Route planner, interactive station map, live guidance

12

How does the project achieve real-time performance?

Theory

- **YOLOv8n (nano)** — Smallest YOLO variant (~6MB), optimized for speed
- **Threading** — Training, audio, and synthetic dataset generation run in background threads
- **MJPEG streaming** — Video frames are JPEG-encoded and streamed efficiently via HTTP multipart
- **Debounced audio** — Audio cues have a 4-second debounce to prevent overwhelming the TTS engine
- **JPEG quality 80** — Frames are compressed at quality 80 for bandwidth efficiency
- **Verbose=False** — YOLO inference runs silently to reduce I/O overhead

13

What security measures are implemented?

Code

- **Secret Key:** Flask app uses `SECRET_KEY` from environment variable or default
- **File Upload Safety:** `secure_filename()` from Werkzeug sanitizes uploaded filenames
- **Allowed Extensions:** Only specific image (`png, jpg, jpeg, bmp, webp`) and video (`mp4, avi, mov, mkv, webm`) types are accepted
- **Max Content Length:** Uploads limited to 500 MB (`MAX_CONTENT_LENGTH = 500 * 1024 * 1024`)
- **Timestamped filenames:** Uploads are prefixed with Unix timestamp to prevent overwrites
- **Local processing:** All data is processed locally — no data sent to external servers

14

Explain how the COCO-to-Navigation class mapping works in demo.py.

Code

Since the pretrained YOLOv8n model detects 80 COCO classes, the demo maps relevant COCO classes to navigation-specific classes:

```
COCO_TO_NAV = {
    'person':      'person',
    'bicycle':     'person_with_luggage',
    'wheelchair':  'wheelchair_user',
    'handbag':     'person_with_luggage',
    'backpack':    'person_with_luggage',
    'suitcase':    'person_with_luggage',
    'umbrella':    'blind_person',      # demo approximation
}
```

This allows the demo to work **without custom training**. The COCO "person" class works immediately; other mappings are approximations for demonstration purposes.

15

What is the role of global state variables in app.py?

Code

```
detector      = Detector()      # YOLO model wrapper (shared across requests)
visualizer    = Visualizer()    # Drawing utility
navigator      = Navigator()     # BFS route planner
audio_guide    = AudioGuide()    # TTS engine

camera_active  = False          # Flag to control live camera stream
camera_lock    = threading.Lock() # Thread safety for camera access
current_camera = None           # Current cv2.VideoCapture object
training_status = {             # Training progress tracking
    'running': False, 'progress': 0,
    'log': [], 'done': False, 'error': None
}
```

These globals are shared across all HTTP requests and threads. The `camera_lock` ensures thread-safe access to camera state.

2 YOLOv8 & Object Detection

1 What is YOLO and what does it stand for?

Theory

YOLO stands for "**You Only Look Once**". It is a real-time object detection algorithm that treats detection as a single regression problem, predicting bounding boxes and class probabilities directly from full images in one forward pass.

Unlike two-stage detectors (R-CNN), YOLO processes the entire image at once, making it significantly faster — enabling real-time detection at 30+ FPS.

2 What is YOLOv8 and why is it used in this project?

Theory

YOLOv8 is the latest version by Ultralytics (2023). It is used because:

- **State-of-the-art accuracy** — Best mAP among single-stage detectors
- **Easy API** — Simple Python interface (`from ultralytics import YOLO`)
- **Multiple sizes** — n (nano), s (small), m (medium), l (large), x (extra-large)
- **Anchor-free** — Eliminates anchor box tuning
- **Built-in training** — `model.train()` handles everything
- **COCO pretrained** — Works out-of-the-box for person detection

3

Explain the Detector class in `utils/detector.py` .

Code

The `Detector` class wraps YOLOv8 inference:

```
class Detector:
    def __init__(self):
        self.model = None
        self.conf = 0.40 # confidence threshold
        self.iou = 0.45 # IoU threshold for NMS

    def load(self, model_path):
        # Loads YOLO model, returns (success, message)
        self.model = YOLO(model_path)

    def detect(self, frame):
        # Runs model.predict() on a BGR frame
        results = self.model.predict(frame, conf=self.conf,
                                     iou=self.iou, verbose=False)

        return results[0]

    def format_results(self, results):
        # Converts YOLO Results → list of dicts:
        # {class_id, class_name, confidence, bbox, center}
```

Key methods: `load()` , `detect()` , `format_results()` , `is_loaded()` , `set_conf()` , `set_iou()` .

4

What is confidence threshold and IoU threshold?

Theory

Confidence Threshold (conf = 0.40): Minimum probability score a detection must have to be considered valid. Detections below this score are discarded. A higher threshold reduces false positives but may miss real objects.

IoU Threshold (iou = 0.45): Used in **Non-Maximum Suppression (NMS)**. IoU = Intersection over Union. If two bounding boxes for the same class overlap with $\text{IoU} > 0.45$, the lower-confidence one is suppressed. This prevents duplicate detections.

Formula: $\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$

5 What is Non-Maximum Suppression (NMS)?

Theory

NMS is a post-processing step that removes redundant overlapping bounding boxes:

1. Sort all detections by confidence score (descending)
2. Select the highest-confidence box
3. Remove all other boxes that have IoU > threshold with the selected box
4. Repeat with remaining boxes

In this project, IoU threshold is set to **0.45** — if two boxes overlap more than 45%, the weaker one is removed.

6 What is the YOLO annotation format used in this project?

Theory

YOLO format stores one `.txt` file per image. Each line represents one object:

```
<class_id> <center_x> <center_y> <width> <height>
```

All values are **normalized** (0 to 1) relative to image dimensions:

- `center_x = (x + w/2) / image_width`
- `center_y = (y + h/2) / image_height`
- `width = box_width / image_width`
- `height = box_height / image_height`

Example: `2 0.5123 0.4567 0.1250 0.3200` means class 2 (blind_person), centered at (51.2%, 45.7%), with width 12.5% and height 32% of the image.

7 Explain the `dataset.yaml` configuration file.

[Code](#)

```
path: data/dataset          # root directory
train: images/train         # training images (relative to path)
val:   images/val           # validation images
nc: 6                        # number of classes
names:
  0: person
  1: wheelchair_user
  2: blind_person
  3: person_with_crutches
  4: elderly_person
  5: person_with_luggage
```

This YAML file tells YOLOv8 where to find training/validation images, how many classes exist, and their names. It is passed to `model.train(data='config/dataset.yaml')`.

8 What is the difference between YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x?

[Theory](#)

Model	Parameters	Speed	Accuracy	Use Case
YOLOv8n (nano)	~3.2M	Fastest	Lowest	Edge/mobile, real-time
YOLOv8s (small)	~11.2M	Fast	Good	Best speed/accuracy balance
YOLOv8m (medium)	~25.9M	Moderate	Better	General purpose
YOLOv8l (large)	~43.7M	Slow	High	High accuracy needed
YOLOv8x (extra)	~68.2M	Slowest	Highest	Maximum accuracy

This project uses **YOLOv8n** by default for demo (speed) and recommends **YOLOv8s** for training (best balance).

9 How does the `format_results()` method work?

[Code](#)

```
def format_results(self, results) → list:
    if results is None: return []
    detections = []
    for box in results.bboxes:
        cls_id = int(box.cls[0])          # class index
        name   = results.names.get(cls_id, ...) # class name
        conf   = float(box.conf[0])       # confidence
        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist()) # bbox
        cx, cy = (x1+x2)//2, (y1+y2)//2    # center point
        detections.append({
            'class_id': cls_id, 'class_name': name,
            'confidence': round(conf, 3),
            'bbox': [x1,y1,x2,y2], 'center': [cx,cy]
        })
    return detections
```

It converts raw YOLO tensors into a clean Python list of dictionaries. The center point is used for zone mapping.

10 What is mAP (mean Average Precision) and how is it evaluated?

[Theory](#)

mAP is the primary metric for evaluating object detection models:

- **Precision** = True Positives / (True Positives + False Positives)
- **Recall** = True Positives / (True Positives + False Negatives)
- **AP (Average Precision)** = Area under the Precision-Recall curve for one class
- **mAP** = Mean of AP across all classes

Two key metrics used in `train.py`:

- `mAP@0.5` — AP at IoU threshold 0.5
- `mAP@0.5:0.95` — Average of mAP at IoU thresholds 0.5, 0.55, ..., 0.95 (stricter)

```
metrics = model.val()
print(f"mAP@0.5      : {metrics.box.map50:.4f}")
print(f"mAP@0.5:0.95: {metrics.box.map:.4f}")
```

11

What is transfer learning and how is it used in this project?

Theory

Transfer learning is a technique where a model pretrained on a large dataset (like COCO with 330K images, 80 classes) is fine-tuned on a smaller, domain-specific dataset.

In this project:

1. Start with `yolo8n.pt` (pretrained on COCO) — already knows general features (edges, shapes, textures)
2. Fine-tune on custom 6-class navigation dataset — adapts to wheelchair users, blind persons, etc.
3. Benefits: Much less data needed (100-200 images vs. thousands), faster convergence, better accuracy

```
model = YOLO('yolo8n.pt')      # Load pretrained weights
model.train(data='config/dataset.yaml', epochs=50) # Fine-tune
```

12

What is the difference between `model.predict()` and `model.train()` ?

Code

Method	Purpose	Usage
<code>model.predict()</code>	Run inference (detection) on images/frames	<code>results = model.predict(frame, conf=0.4)</code>
<code>model.train()</code>	Train/fine-tune the model on a dataset	<code>model.train(data='dataset.yaml', epochs=50)</code>
<code>model.val()</code>	Evaluate model on validation set	<code>metrics = model.val()</code>

`predict()` is used at inference time (fast, no gradients), while `train()` updates model weights using backpropagation.

13 What are bounding boxes in object detection?

Theory

A **bounding box** is a rectangle that encloses a detected object. It is defined by:

- **xyxy format:** (x1, y1, x2, y2) — top-left and bottom-right corners (used in `results.bboxes.xyxy`)
- **xywh format:** (center_x, center_y, width, height) — used in YOLO label files

The detector extracts `box.xyxy[0]` for pixel coordinates and computes center as:

```
cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
```

14 What is image size (imgsz) and why is 640 used?

Theory

`imgsz=640` means input images are resized to 640×640 pixels before feeding into the YOLO model. This is because:

- Neural networks require fixed input dimensions
- 640×640 is the default for YOLOv8 — optimized stride/grid alignment
- Larger sizes (1280) improve small object detection but are slower
- Smaller sizes (320) are faster but less accurate

15 What is batch size in training?

Theory

Batch size is the number of images processed simultaneously in one training iteration. Default is `batch=16`.

- **Larger batch** — More stable gradients, faster training (needs more GPU memory)
- **Smaller batch** — Less memory, more noise in gradients, may generalize better
- **batch=-1** — Auto-batch: YOLO automatically determines the largest batch that fits in GPU memory

16

What is an epoch in training?

Theory

An **epoch** is one complete pass through the entire training dataset. If you have 200 training images and batch size 16, one epoch = $200/16 = 13$ iterations (batches).

The project defaults to **50 epochs**. More epochs allow the model to learn better, but too many can cause **overfitting** (memorizing training data instead of generalizing).

17

What data augmentation techniques are used?

Code

When `--augment` flag is used in `train.py`, extra augmentations are applied:

```
train_kwargs.update(dict(
    hsv_h=0.015,      # Hue shift ±1.5%
    hsv_s=0.7,        # Saturation shift ±70%
    hsv_v=0.4,        # Value/brightness shift ±40%
    degrees=10,       # Random rotation ±10°
    translate=0.1,    # Random translation ±10%
    scale=0.5,        # Random scaling ±50%
    flipud=0.1,       # Vertical flip 10% probability
    fliplr=0.5,       # Horizontal flip 50% probability
    mosaic=1.0,       # Mosaic augmentation 100%
))
```

Mosaic augmentation combines 4 training images into one, exposing the model to more variety and context.

18

What is the COCO dataset?

Theory

COCO (Common Objects in Context) is a large-scale object detection dataset with:

- 330K images, 200K labeled
- 80 object categories (person, car, dog, chair, etc.)
- 1.5 million object instances

YOLOv8n.pt is pretrained on COCO, which means it can detect "person" out-of-the-box. The demo.py maps COCO's "person" class directly to the navigation system's "person" class.

19

What is the difference between one-stage and two-stage detectors?

Theory

Feature	One-Stage (YOLO, SSD)	Two-Stage (R-CNN, Faster R-CNN)
Approach	Single forward pass	Region proposal + classification
Speed	Very fast (real-time)	Slower
Accuracy	Slightly lower (but YOLOv8 is competitive)	Higher (traditionally)
Use Case	Real-time applications	High-accuracy applications

This project uses YOLO (one-stage) because **real-time speed is critical** for navigation assistance.

20

What is anchor-free detection in YOLOv8?

Theory

Earlier YOLO versions (v3, v5) used **anchor boxes** — predefined box shapes that the model adjusts. YOLOv8 is **anchor-free**, meaning:

- It directly predicts the center point and dimensions of objects
- No need to predefine anchor sizes/ratios
- Simplifies training and reduces hyperparameter tuning
- Better generalization to objects of unusual sizes

3 Flask Web Application & REST API

1 What is Flask and why is it used?

Theory

Flask is a lightweight Python web framework (micro-framework). It is used because:

- Minimal boilerplate — easy to set up
- Built-in development server with hot reload
- Jinja2 templating for HTML rendering
- Easy to create REST APIs with JSON responses
- Good for prototyping and small-to-medium applications
- Threading support for concurrent requests

```
app = Flask(__name__)  
app.run(debug=True, threaded=True, host='0.0.0.0', port=5000)
```

2

List all the API endpoints in the application.

Code

Method	Endpoint	Description
POST	<code>/api/load_model</code>	Load a YOLO model
POST	<code>/api/train</code>	Start model training
GET	<code>/api/train/status</code>	Poll training progress
POST	<code>/api/train/stop</code>	Stop training
POST	<code>/api/detect/image</code>	Detect objects in uploaded image
POST	<code>/api/detect/video</code>	Process uploaded video
GET	<code>/video_feed</code>	MJPEG live camera stream
POST	<code>/api/camera/stop</code>	Stop live camera
POST	<code>/api/navigation/route</code>	BFS route between zones
GET	<code>/api/navigation/map</code>	Station map JSON
POST	<code>/api/speak</code>	Trigger TTS
GET	<code>/api/models</code>	List saved models
POST	<code>/api/models/upload</code>	Upload .pt model
POST	<code>/api/dataset/synthetic</code>	Generate synthetic dataset
POST	<code>/api/dataset/roboflow</code>	Download Roboflow dataset
POST	<code>/api/upload_dataset</code>	Upload dataset .zip
POST	<code>/api/upload_yaml</code>	Upload dataset YAML

3 Explain the `/api/detect/image` endpoint flow.

[Code](#)

1. Receive uploaded image file via `request.files.get('image')`
2. Validate file type and save with `save_upload()` (timestamped filename)
3. Check if model is loaded (`detector.is_loaded()`)
4. Read image with `cv2.imread(path)`
5. Run detection: `results = detector.detect(frame)`
6. Draw annotations: `annotated = visualizer.draw(frame, results, navigator)`
7. Save annotated image: `cv2.imwrite(out_path, annotated)`
8. Format detections and compute navigation directions
9. Return JSON with result image path, detections list, directions, and count

4 How does MJPEG live streaming work in the `/video_feed` endpoint?

[Code](#)

MJPEG (Motion JPEG) streaming sends continuous JPEG frames over HTTP using **multipart/x-mixed-replace**:

```
def generate_frames(destination='platform_1'):
    cap = cv2.VideoCapture(0)          # Open webcam
    while camera_active:
        ret, frame = cap.read()
        # ... detection & annotation ...
        ret2, buffer = cv2.imencode('.jpg', annotated,
                                    [cv2.IMWRITE_JPEG_QUALITY, 80])

        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' +
               buffer.tobytes() + b'\r\n')

@app.route('/video_feed')
def video_feed():
    return Response(generate_frames(destination),
                    mimetype='multipart/x-mixed-replace; boundary=frame')
```

The browser receives this as a continuous stream of JPEG images, creating a video effect. Each frame is separated by the `--frame` boundary.

5 What is `secure_filename()` and why is it important?

[Code](#)

`secure_filename()` from `werkzeug.utils` sanitizes user-provided filenames to prevent:

- **Path traversal attacks:** e.g., `../../etc/passwd` → `etc_passwd`
- **Special characters:** Removes shell-dangerous characters
- **Unicode issues:** Normalizes Unicode characters

```
from werkzeug.utils import secure_filename
fname = secure_filename(file.filename) # "../../hack.py" → "hack.py"
```

6 How does training work via the web API?

[Code](#)

Training runs in a **background thread** so it doesn't block the web server:

```
@app.route('/api/train', methods=['POST'])
def start_training():
    # 1. Check if already training
    if training_status['running']:
        return error

    # 2. Parse parameters (epochs, batch, model, etc.)
    # 3. Reset training_status dict
    # 4. Start background thread:
    t = threading.Thread(target=run_training, daemon=True)
    t.start()

    # 5. run_training() function:
    # - Loads YOLO model
    # - Calls model.train(...)
    # - Copies best.pt to models/
    # - Updates training_status dict
```

The frontend polls `/api/train/status` to track progress, and `/api/train/stop` can halt it.

7 What is the `allowed_file()` function?

Code

```
ALLOWED_IMAGES = {'png', 'jpg', 'jpeg', 'bmp', 'webp'}
ALLOWED_VIDEOS = {'mp4', 'avi', 'mov', 'mkv', 'webm'}

def allowed_file(filename, allowed):
    return '.' in filename and \
           filename.rsplit('.', 1)[1].lower() in allowed
```

It extracts the file extension using `rsplit('.', 1)` (split from right, max 1 split) and checks if it's in the allowed set. This prevents uploading malicious file types.

8 What is Jinja2 templating?

Theory

Jinja2 is Flask's default template engine. It allows embedding Python logic in HTML:

```
<!-- templates/index.html -->
{% extends "base.html" %}      <!-- Template inheritance -->

{% for model in models %}      <!-- Loop -->
    <option>{{ model }}</option> <!-- Variable -->
{% endfor %}

{% if models %}                <!-- Conditional -->
    <p>Models available</p>
{% endif %}
```

Flask's `render_template()` combines templates with data:

```
return render_template('index.html', models=models)
```

9

What is `jsonify()` in Flask?

Code

`jsonify()` converts Python dictionaries/lists to JSON HTTP responses with proper `Content-Type: application/json` header:

```
return jsonify({
    'success': True,
    'detections': detections,
    'count': len(detections)
})
```

It handles serialization, sets headers, and returns a proper Flask Response object.

10

What is the purpose of `app.config['MAX_CONTENT_LENGTH']` ?

Code

```
app.config['MAX_CONTENT_LENGTH'] = 500 * 1024 * 1024 # 500 MB
```

This limits the maximum upload size to 500 MB. If a user tries to upload a file larger than this, Flask automatically returns a **413 Request Entity Too Large** error. This prevents the server from running out of memory or disk space from excessively large uploads.

11

What is the purpose of `threading.Lock()` in the camera handling?

Code

```
camera_lock = threading.Lock()

with camera_lock:
    camera_active = True
    current_camera = cap
```

A **Lock** prevents race conditions when multiple threads access shared resources (camera state). Only one thread can hold the lock at a time. Without it, the camera could be started/stopped simultaneously from different requests, causing crashes.

12

How does the dataset upload endpoint work?

Code

```
@app.route('/api/upload_dataset', methods=['POST'])
def upload_dataset():
    f = request.files.get('dataset')
    if f and f.filename.endswith('.zip'):
        zpath = os.path.join('data', 'dataset.zip')
        f.save(zpath)                # Save zip
        import zipfile
        with zipfile.ZipFile(zpath, 'r') as z:
            z.extractall('data/dataset') # Extract to dataset dir
        os.remove(zpath)              # Clean up zip
        return jsonify({'success': True, ...})
```

It accepts a `.zip` file, saves it, extracts contents into `data/dataset/`, and deletes the zip.

13

What
does`app.run(debug=True, threaded=True, host='0.0.0.0',
port=5000)`

mean?

Code

- `debug=True` — Enables auto-reload on code changes + detailed error pages
- `threaded=True` — Handles each request in a separate thread (needed for video streaming)
- `host='0.0.0.0'` — Listens on all network interfaces (accessible from other devices)
- `port=5000` — Runs on port 5000

Note: `debug=True` should be disabled in production for security reasons.

14

What is a REST API?

Theory

REST (Representational State Transfer) is an architectural style for web APIs:

- **Resources** are identified by URLs (`/api/models` , `/api/detect/image`)
- **HTTP methods** define actions: GET (read), POST (create/action), PUT (update), DELETE (remove)
- **Stateless** — Each request contains all needed information
- **JSON** is the standard data format

This project follows REST conventions: GET for fetching data, POST for actions/uploads.

15

How does the `save_upload()` helper work?

Code

```
def save_upload(file, allowed_set):
    if file and allowed_file(file.filename, allowed_set):
        fname = secure_filename(file.filename) # sanitize
        ts    = str(int(time.time()))          # unix timestamp
        fname = f"{ts}_{fname}"                # prevent collisions
        path  = os.path.join(UPLOAD_FOLDER, fname)
        file.save(path)
        return path
    return None
```

It validates the file, sanitizes the name, adds a timestamp prefix for uniqueness, saves to `static/uploads/` , and returns the file path.

16

What is the difference between `request.json` and `request.files` ?

Code

Property	Usage	Content-Type
<code>request.json</code>	Parse JSON body data	<code>application/json</code>
<code>request.files</code>	Access uploaded files	<code>multipart/form-data</code>
<code>request.form</code>	Access form fields	<code>application/x-www-form-urlencoded</code>
<code>request.args</code>	URL query parameters	N/A (in URL)

Example: `/api/train` uses `request.json` for parameters; `/api/detect/image` uses `request.files` for the image.

17

What is the purpose of the daemon thread in training?

Code

```
t = threading.Thread(target=run_training, daemon=True)
t.start()
```

`daemon=True` means the thread will be automatically killed when the main program exits. Without it, the training thread would keep the application alive even after you try to stop the server. Daemon threads are used for background tasks that should not prevent program shutdown.

```
cap = cv2.VideoCapture(path)
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # codec
out = cv2.VideoWriter(out_path, fourcc, fps, (w, h))

while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break
    results = detector.detect(frame) # detect per frame
    annotated = visualizer.draw(frame, results, navigator)
    out.write(annotated) # write annotated frame

cap.release()
out.release()
```

It opens the video, processes each frame through YOLO detection and visualization, writes the annotated frame to a new MP4 file, and returns the result path.

4 BFS Navigation & Graph Routing

1

What is BFS (Breadth-First Search)?

Theory

BFS is a graph traversal algorithm that explores nodes level by level, starting from the source. It guarantees finding the **shortest path** in an unweighted graph.

- Uses a **queue** (FIFO) data structure
- Time complexity: $O(V + E)$ where V = vertices, E = edges
- Space complexity: $O(V)$
- Explores all neighbors at current depth before moving deeper

```
def get_route(self, src, dst):
    if src == dst:
        return [src], ["You have arrived at your destination."]

    from collections import deque
    visited = {src: None}          # Track parent of each node
    queue = deque([src])          # BFS queue

    while queue:
        node = queue.popleft()     # Dequeue front
        if node == dst:
            break                  # Found destination
        for (nxt, _, _) in STATION_GRAPH.get(node, []):
            if nxt not in visited:
                visited[nxt] = node # Record parent
                queue.append(nxt)   # Enqueue neighbor

    # Reconstruct path by backtracking parents
    path = []
    cur = dst
    while cur is not None:
        path.append(cur)
        cur = visited.get(cur)
    path.reverse()

    # Build step-by-step instructions
    steps = []
    for i in range(len(path) - 1):
        for (nxt, instr, _) in STATION_GRAPH.get(path[i], []):
            if nxt == path[i+1]:
                steps.append(instr)
                break
    return path, steps
```

3

What data structure is used to represent the station graph?

Code

The station is represented as an **adjacency list** using a Python dictionary:

```
STATION_GRAPH = {
    'entrance': [
        ('main_hall', 'Walk straight into the main hall', 'down'),
        ('ticket_counter', 'Turn right to reach ticket counter', 'right'),
        ('restroom', 'Turn left for the restroom', 'left'),
    ],
    'main_hall': [
        ('entrance', 'Walk towards the main entrance', 'up'),
        ('platform_1', 'Turn left to Platform 1', 'left'),
        ...
    ],
    ...
}
```

Each key is a zone, and the value is a list of tuples: `(neighbor_zone, instruction_text, arrow_direction)`. This is a **directed graph** where each edge has navigation metadata.

4

How does zone detection work from pixel coordinates?

Code

```
def get_zone(self, fx: float, fy: float) → str:
    """Return zone name for a normalised (fx, fy) position."""
    for zone, (x1, y1, x2, y2) in Settings.ZONES.items():
        if x1 ≤ fx ≤ x2 and y1 ≤ fy ≤ y2:
            return zone
    return 'main_hall' # default fallback
```

Steps:

1. Detection center (cx, cy) is computed from bounding box
2. Normalized: `fx = cx / frame_width`, `fy = cy / frame_height` (values 0 to 1)
3. Compare against each zone's fractional bounds
4. First matching zone is returned; default is 'main_hall'

5

How is path reconstruction done after BFS?

Theory

BFS stores the **parent** of each visited node in the **visited** dictionary. To reconstruct the path:

1. Start at destination node
2. Follow parent pointers backward: `cur = visited.get(cur)`
3. Collect each node into a list
4. Reverse the list to get source-to-destination order

Example: entrance → main_hall → platform_1

```
visited = {'entrance': None, 'main_hall': 'entrance', 'platform_1': 'main_hall'}  
# Backtrack from platform_1: platform_1 → main_hall → entrance  
# Reverse: [entrance, main_hall, platform_1]
```

6

What happens if no route is found?

Code

```
if path[0] != src:  
    return [src, dst], [f"Direct route not found. Head towards {dst_name}."]
```

If backtracking doesn't reach the source (disconnected graph), the system returns a direct fallback instruction: "Head towards [destination]". This ensures the user always gets *some* guidance even if the graph is incomplete.

7 Explain the `get_directions_from_detections()` method.

[Code](#)

```
def get_directions_from_detections(self, detections, frame_shape,
                                   destination='platform_1'):
    h, w = frame_shape[:2]
    directions = []
    for det in detections:
        cx, cy = det['center']
        fx, fy = cx / w, cy / h          # Normalize
        zone = self.get_zone(fx, fy)      # Find zone
        _, steps = self.get_route(zone, destination) # BFS
        directions.append({
            'class': det['class_name'],
            'zone': ZONE_DISPLAY_NAMES.get(zone, zone),
            'instruction': steps[0],      # First step only
            'confidence': det['confidence'],
        })
    return directions
```

For each detected person, it computes their zone, runs BFS to get the route, and returns the first step instruction for audio/visual guidance.

8 What is the `get_arrow_direction()` method used for?

[Code](#)

It determines the arrow direction to display on the annotated frame:

1. If current zone = destination → return `'arrived'`
2. Check direct neighbors for the destination
3. If not a direct neighbor, run BFS and use the first step's arrow direction
4. Fallback: `'up'`

The arrow is drawn visually on the frame near the detected person, pointing them toward their next zone.

9

Why is `deque` used instead of a regular list for BFS?

Theory

`collections.deque` provides $O(1)$ `popleft()` operations, while a regular list's `pop(0)` is $O(n)$ because all elements must be shifted. For BFS:

Operation	list	deque
append (enqueue)	$O(1)$	$O(1)$
pop(0) / popleft (dequeue)	$O(n)$	$O(1)$

This makes deque essential for efficient BFS in larger graphs.

10

How does the station map JSON endpoint work?

Code

```
def get_station_map(self):
    nodes = []
    for zone, name in ZONE_DISPLAY_NAMES.items():
        x1, y1, x2, y2 = Settings.ZONES[zone]
        nodes.append({
            'id': zone, 'name': name,
            'x': (x1+x2)/2, 'y': (y1+y2)/2, # center coords
        })
    edges = []
    for src, connections in STATION_GRAPH.items():
        for (dst, instr, _) in connections:
            edges.append({'from': src, 'to': dst, 'instruction': instr})
    return {'nodes': nodes, 'edges': edges, 'zone_names': ZONE_DISPLAY_NAMES}
```

Returns nodes (zones with center positions) and edges (connections with instructions) as JSON for the frontend to render an interactive station map.

11

Why is BFS preferred over DFS for route finding?

Theory

- **BFS guarantees shortest path** in unweighted graphs — critical for giving passengers the most efficient route
- DFS might find a valid path but not the shortest one
- Station graph is small (9 nodes), so BFS performance overhead is negligible
- BFS explores level by level, which naturally gives the minimum number of zone transitions

12

What is the time complexity of BFS in this station graph?

Theory

BFS time complexity is $O(V + E)$ where V = number of zones (vertices) and E = number of connections (edges).

- $V = 9$ zones
- $E \approx 25$ connections (edges in STATION_GRAPH)
- So $O(9 + 25) = O(34)$ — essentially constant time

Even with path reconstruction, the total work is negligible for 9 nodes, making it suitable for real-time per-frame computation.

13

What are the 8 possible arrow directions?

Code

```
ARROW_MAP = {
    'left':      (-60,  0),
    'right':     ( 60,  0),
    'up':        (  0, -60),
    'down':      (  0, 60),
    'up-left':   (-45, -45),
    'up-right':  ( 45, -45),
    'down-left': (-45, 45),
    'down-right': ( 45, 45),
    'arrived':   (  0,  0),  # Special: draw circle instead
}
```

Each direction maps to (dx, dy) pixel offsets from the detection center. The arrow is drawn using `cv2.arrowedLine()`. The 'arrived' state draws a green circle with "HERE" text.

14

What is the role of `ZONE_DISPLAY_NAMES` ?[Code](#)

```
ZONE_DISPLAY_NAMES = {  
    'entrance':      'Entrance',  
    'main_hall':     'Main Hall',  
    'platform_1':    'Platform 1',  
    'restroom':      'Restroom / Washroom',  
    'elevator':      'Elevator / Lift',  
    ...  
}
```

It maps internal zone keys (snake_case) to **human-readable display names**. Used in API responses, audio announcements, and UI display. Internal keys are used for graph operations while display names are user-facing.

15

Can the station graph handle bidirectional connections?

[Code](#)

Yes. The graph is **directed** — each direction has its own instruction. For example:

- `entrance → main_hall` : "Walk straight into the main hall" (arrow: down)
- `main_hall → entrance` : "Walk towards the main entrance" (arrow: up)

Bidirectional connections are explicitly defined with different instructions and arrows for each direction, providing context-appropriate navigation guidance.

5 OpenCV & Visualization

1 What is OpenCV and what is it used for in this project?

Theory

OpenCV (Open Source Computer Vision Library) is an open-source library for image and video processing. In this project it is used for:

- Reading images (`cv2.imread()`)
- Capturing video from webcam (`cv2.VideoCapture()`)
- Drawing bounding boxes, arrows, text overlays (`cv2.rectangle()` , `cv2.arrowedLine()`)
- Encoding frames to JPEG for streaming (`cv2.imencode()`)
- Writing output videos (`cv2.VideoWriter()`)
- Displaying windows (`cv2.imshow()`)

2 Explain the Visualizer class and its methods.

Code

Method	Purpose
<code>draw()</code>	Main entry — draws zones, HUD, boxes, arrows
<code>_draw_box()</code>	Draws bounding box with corner accents and label
<code>_draw_zone_overlay()</code>	Semi-transparent colored zone rectangles
<code>_draw_hud()</code>	Top bar with system title
<code>_draw_detection_count()</code>	Bottom bar showing person count
<code>_draw_arrow()</code>	Navigation arrow from detection center

3 How does the semi-transparent zone overlay work?

Code

```
def _draw_zone_overlay(self, img, w, h):
    overlay = img.copy() # Copy of original
    for zone, (x1f, y1f, x2f, y2f) in Settings.ZONES.items():
        zx1, zy1 = int(x1f * w), int(y1f * h) # Convert fractions to pixels
        zx2, zy2 = int(x2f * w), int(y2f * h)
        cv2.rectangle(overlay, (zx1,zy1), (zx2,zy2), color, -1) # Filled rect
        cv2.rectangle(img, (zx1,zy1), (zx2,zy2), color, 1) # Border
        cv2.addWeighted(overlay, 0.12, img, 0.88, 0, img) # Blend at 12% opacity
```

`cv2.addWeighted()` blends two images: $dst = \alpha \times overlay + \beta \times img + \gamma$. With $\alpha=0.12$, the colored rectangles appear as subtle transparent overlays.

4 What is BGR color format and why does OpenCV use it?

Theory

OpenCV uses **BGR (Blue, Green, Red)** instead of the more common RGB order. This is a historical decision from when the library was created — early camera hardware used BGR format.

Example from settings: `'person': (219, 86, 26)` means B=219, G=86, R=26 (appears orange-brown).

When displaying or saving, OpenCV handles BGR natively. When converting to PIL or web formats, use `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

5 How do corner accents enhance bounding box visualization?

[Code](#)

```
def _draw_box(self, img, x1, y1, x2, y2, label, conf, color):
    cv2.rectangle(img, (x1,y1), (x2,y2), color, 2) # Main box
    cs = 14 # Corner size
    # Top-left corner
    cv2.line(img, (x1,y1), (x1+cs, y1), color, 3) # horizontal
    cv2.line(img, (x1,y1), (x1, y1+cs), color, 3) # vertical
    # ... same for other 3 corners ...
```

Corner accents are **thicker short lines (3px, 14px long)** at each corner of the bounding box. This creates a modern, professional-looking detection visualization commonly seen in security/surveillance interfaces.

6 What is `cv2.VideoWriter_fourcc()` ?

[Code](#)

```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
writer = cv2.VideoWriter(out_path, fourcc, fps, (w, h))
```

FourCC (Four Character Code) is a 4-byte code that identifies the video codec:

- `'mp4v'` — MPEG-4 codec (used in this project)
- `'XVID'` — Xvid MPEG-4
- `'MJPG'` — Motion JPEG
- `'H264'` — H.264 codec

The `*'mp4v'` unpacks the string into individual characters: `'m', 'p', '4', 'v'`.

7 How is FPS calculated in the live detection mode?

[Code](#)

```
t0 = time.time()
results = detector.detect(frame)
annotated = visualizer.draw(frame, results, ...)
elapsed = time.time() - t0
fps_disp = 1 / max(elapsed, 0.001) # Prevent division by zero
```

FPS = 1 / processing_time_per_frame. The `max(elapsed, 0.001)` ensures no division by zero. In the demo, exponential smoothing is used: `fps = 0.9*fps + 0.1*(1/elapsed)` for stable display.

8 What does `cv2.waitKey(1) & 0xFF` do?

[Code](#)

`cv2.waitKey(1)` waits 1 millisecond for a key press. The `& 0xFF` masks the result to get the last 8 bits (ASCII value), needed for compatibility on some systems where `waitKey` returns a larger integer.

```
key = cv2.waitKey(1) & 0xFF
if key == ord('q'): # 'q' to quit
    break
if key == ord('s'): # 's' to save snapshot
    cv2.imwrite(snap, annotated)
```

9 How does `cv2.imencode()` work for MJPEG streaming?

[Code](#)

```
ret2, buffer = cv2.imencode('.jpg', annotated,
                             [cv2.IMWRITE_JPEG_QUALITY, 80])
```

It compresses an OpenCV image array into JPEG format in memory (no file I/O). The second argument specifies JPEG quality (0-100, default 95). Quality 80 is used for streaming to reduce bandwidth while maintaining visual quality. Returns a boolean success flag and the compressed byte buffer.

10

What is the difference between `cv2.imread()` and `cv2.VideoCapture()` ?

Theory

Function	Purpose	Returns
<code>cv2.imread(path)</code>	Read a single image from disk	NumPy array (BGR)
<code>cv2.VideoCapture(source)</code>	Open video file or camera	VideoCapture object

VideoCapture can open a camera (`VideoCapture(0)` for webcam index 0), a video file (`VideoCapture('video.mp4')`), or an RTSP stream. It reads frames one by one with `cap.read()` .

11

What is `cv2.putText()` and what parameters does it take?

Code

```
cv2.putText(img,                # image to draw on
            'Navigation Assistance', # text string
            (8, 19),            # position (x, y) of text baseline
            cv2.FONT_HERSHEY_SIMPLEX, # font face
            0.50,                # font scale
            (180, 230, 255),    # color (BGR)
            1,                  # thickness
            cv2.LINE_AA)        # anti-aliased line type
```

`cv2.LINE_AA` enables anti-aliasing for smoother text rendering.

12

How is the camera resolution set in the project?

Code

```
cap = cv2.VideoCapture(0)                # Open default webcam
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)  # Set width to 1280
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)  # Set height to 720
```

`cap.set()` configures camera properties. `CAP_PROP_FRAME_WIDTH` and `CAP_PROP_FRAME_HEIGHT` set the capture resolution to 720p (1280×720). The actual resolution depends on camera hardware support.

6 Audio Guide & Text-to-Speech

1 What TTS engines does the AudioGuide use?

Theory

The AudioGuide uses a **fallback chain** of 3 TTS methods:

1. **pyttsx3** (primary) — Offline TTS engine. Works without internet. Uses OS-native speech (SAPI5 on Windows, espeak on Linux, NSSpeech on macOS).
2. **gTTS** (fallback) — Google Text-to-Speech. Requires internet. Generates MP3 files played via `playsound`.
3. **Console print** (last resort) — If both TTS engines fail, prints the message to console.

2 Explain the `speak()` method in AudioGuide.

Code

```
def speak(self, text: str):
    if not text: return
    with self._lock:
        # Thread-safe
        if self._engine:
            # Try pyttsx3
            try:
                self._engine.say(text)
                self._engine.runAndWait()
                return
            except: pass
        try:
            # Fallback: gTTS
            from gtts import gTTS
            tts = gTTS(text=text, lang='en', slow=False)
            with tempfile.NamedTemporaryFile(delete=False, suffix='.mp3') as f:
                tmp = f.name
            tts.save(tmp)
            playsound.playsound(tmp)
            os.unlink(tmp)
            # Delete temp file
        except:
            print(f"[AudioGuide] {text}") # Last resort
```

3

What is debouncing and how is it used in `speak_async()` ?

Code

```
def speak_async(self, text: str, debounce: float = 4.0):
    now = time.time()
    if text == self._last_msg and (now - self._last_time) < debounce:
        return # Skip if same message within 4 seconds
    self._last_msg = text
    self._last_time = now
    t = threading.Thread(target=self.speak, args=(text,), daemon=True)
    t.start()
```

Debouncing prevents the same audio message from being repeated too frequently. If the same instruction is triggered within 4 seconds, it is silently ignored. This prevents overlapping speech when detecting the same person across multiple frames.

4

Why is `threading.Lock()` used in the AudioGuide?

Code

The `self._lock` ensures that only one thread can use the TTS engine at a time. Without it:

- Multiple detection frames might trigger concurrent speech calls
- `pyttsx3` is not thread-safe — concurrent access crashes it
- Two threads writing to the same audio output creates garbled sound

`with self._lock:` is a context manager that automatically acquires and releases the lock.

5 What are the predefined audio announcements?

[Code](#)

```
def announce_destination(self, destination_name):
    self.speak_async(f"Setting destination to {destination_name}")

def announce_direction(self, instruction):
    self.speak_async(instruction)

def announce_arrived(self):
    self.speak_async("You have arrived at your destination.")

def welcome(self):
    self.speak_async(
        "Welcome to the Navigation Assistance System. "
        "Please select your destination to begin."
    )
```

6 How does pyttsx3 differ from gTTS?

[Theory](#)

Feature	pyttsx3	gTTS
Internet Required	No (offline)	Yes (online)
Quality	Moderate (robotic)	High (natural)
Speed Control	Yes (rate property)	Yes (slow parameter)
Output	Direct audio playback	Saves MP3 file
Platforms	Windows, Linux, macOS	Any (uses Google API)
Latency	Low (immediate)	Higher (network + file I/O)

7 How is TTS speech rate configured?

[Code](#)

```
self._engine = pyttsx3.init()
self._engine.setProperty('rate', 140)    # Words per minute (default ~200)
self._engine.setProperty('volume', 1.0)  # Volume 0.0 to 1.0
```

Rate 140 WPM is slower than default (200) for clearer comprehension by passengers who may be elderly or have hearing difficulties.

8 How does the audio cue timing work during live camera streaming?

[Code](#)

```
# In app.py generate_frames():
if int(time.time()) % 3 == 0 and detections:
    dirs = navigator.get_directions_from_detections(...)
    if dirs:
        audio_guide.speak_async(dirs[0]['instruction'])
```

Audio is triggered every 3 seconds (when Unix timestamp is divisible by 3). Combined with the 4-second debounce in `speak_async()`, this creates natural-feeling periodic announcements without overwhelming the user.

9 What happens if the pyttsx3 engine fails to initialize?

[Code](#)

```
def _init_engine(self):
    try:
        import pyttsx3
        self._engine = pyttsx3.init()
        self._engine.setProperty('rate', 140)
    except Exception:
        self._engine = None    # Graceful degradation
```

The engine is set to `None`, and subsequent `speak()` calls automatically fall back to gTTS or console output. This ensures the system works even if pyttsx3 is not installed or fails on the platform.

10

Why is `tempfile.NamedTemporaryFile` used in gTTS fallback?

Code

```
with tempfile.NamedTemporaryFile(delete=False, suffix='.mp3') as f:
    tmp = f.name
    tts.save(tmp)
    playsound.playsound(tmp)
    os.unlink(tmp)          # Delete after playing
```

gTTS generates MP3 files (it can't play audio directly). `NamedTemporaryFile` creates a unique temporary file. `delete=False` keeps the file open for playsound. After playback, `os.unlink()` deletes the file to avoid accumulating temp files.

7 Dataset Preparation & Synthetic Data

1 What is the YOLO dataset folder structure?

Theory

```
data/dataset/  
├─ images/  
│   ├─ train/    # Training images (.jpg, .png)  
│   └─ val/      # Validation images  
└─ labels/  
    ├─ train/    # YOLO .txt labels (one per image)  
    └─ val/      # Validation labels
```

Each label file has the **same name** as its image (different extension). Example: `photo001.jpg`
→ `photo001.txt` .

2 How does the synthetic dataset generator work?

Code

`gen_synthetic.py` creates training data without real photos using **PIL (Pillow)**:

1. Creates a 640×640 image with a station background (floor, ceiling, pillars, signs)
2. Randomly places 1-5 person silhouettes per image
3. Each person has class-specific features (wheelchair wheels, white cane, crutches, luggage)
4. Adds shadows, noise, and optional blur for realism
5. Generates YOLO-format labels with normalized coordinates
6. Saves images as JPEG (quality 88) and labels as .txt

```
python download_dataset.py --dataset synthetic --train 200 --val 50
```

3

How are YOLO label coordinates calculated in synthetic generation?

[Code](#)

```
# Pixel coordinates of drawn person
px, py = random position
pw, ph = random width, height

# Convert to YOLO normalized format
cx = (px + pw / 2) / W      # center_x (0 to 1)
cy = (py + ph / 2) / H      # center_y (0 to 1)
nw = pw / W                 # normalized width
nh = ph / H                 # normalized height

# Write label line
labels.append(f"{cls_id} {cx:.4f} {cy:.4f} {nw:.4f} {nh:.4f}")
```

All values are divided by image dimensions (W=640, H=640) to get fractions between 0 and 1.

4

What is the `verify_dataset()` function?

[Code](#)

It checks dataset integrity by verifying:

1. **Missing labels:** Every image must have a corresponding .txt label file
2. **Orphan labels:** Every label must have a corresponding image
3. **Class distribution:** Counts instances of each class across all labels

```
python prepare_dataset.py --verify
```

Outputs a summary with total images, issues found, and a bar chart of class distribution.

5 How does automatic train/val splitting work?

[Code](#)

```
def split_dataset(source, split=0.8):
    images = sorted(list(src.glob('*.*jpg')) + ...) # Find all images
    random.shuffle(images)                         # Random shuffle
    n_train = int(len(images) * split)              # 80% train
    splits = {
        'train': images[:n_train],
        'val':   images[n_train:]                  # 20% val
    }
    # Copy images and matching labels to respective folders
```

Default 80/20 split. Images are randomly shuffled to ensure representative distribution.

6 What dataset sources are supported?

[Code](#)

Source	Command	Requirements
Synthetic (generated)	<code>--dataset synthetic</code>	None (PIL/Pillow)
Roboflow API	<code>--dataset roboflow --api-key KEY</code>	Roboflow API key
Open Images (fiftyone)	<code>--dataset openimages</code>	fiftyone library
Manual upload	Web UI → Train page	.zip file
Custom images	<code>prepare_dataset.py --source path</code>	Annotated images

7

How does the Roboflow download work?

[Code](#)

```
from roboflow import Roboflow
rf = Roboflow(api_key=api_key)
project = rf.workspace(workspace).project(project)
dataset = project.version(version).download("yolov8", location=str(DATASET_DIR))
```

The Roboflow SDK connects to their API, downloads the specified project version in YOLOv8 format, and saves it directly into the dataset directory. This provides real, annotated training data.

8

What is the `extract_and_organise()` function?

[Code](#)

It handles uploaded zip files by:

1. Extracting the zip to a temporary directory
2. Auto-detecting YOLO structure (finding images, labels, yaml)
3. Detecting train/val split folders, or auto-splitting 80/20 if flat
4. Copying files to the standard dataset directory structure
5. Saving any found YAML config to `config/`
6. Cleaning up temporary files

9

What class-specific visual features are drawn for each passenger type in synthetic data?

[Code](#)

Class	Visual Feature
person (0)	Basic head + body + legs silhouette
wheelchair_user (1)	Wheel ellipses below body
blind_person (2)	White cane line extending from hand
person_with_crutches (3)	Two diagonal crutch lines
elderly_person (4)	Walking stick / cane
person_with_luggage (5)	Rectangular luggage box beside body

10

Why is the minimum recommended dataset size 100-200 images per class?

[Theory](#)

Deep learning models need sufficient diverse examples to learn generalizable features:

- **Too few images (<50)** — Model overfits; memorizes training data instead of learning patterns
- **100-200 images** — Minimum for reasonable performance with transfer learning (pretrained YOLO)
- **500+ images** — Better generalization and robustness
- **1000+ images** — Ideal for production-quality models

Data augmentation (flipping, rotation, scaling) helps artificially increase effective dataset size.

11

What is `ImageFilter.GaussianBlur` used for in synthetic generation?

[Code](#)

```
if random.random() < 0.3:  
    img = img.filter(ImageFilter.GaussianBlur(radius=0.8))
```

A slight Gaussian blur is randomly applied to 30% of synthetic images to simulate camera defocus and add realism. This makes the model more robust to blurry real-world images.

12

How does `write_yaml()` auto-generate the dataset configuration?[Code](#)

```
def write_yaml(classes):
    yaml_content = f"""path: data/dataset
train: images/train
val:   images/val
nc: {len(classes)}
names:
"""
    for i, c in enumerate(classes):
        yaml_content += f"  {i}: {c}\n"

    with open('config/dataset.yaml', 'w') as f:
        f.write(yaml_content)
```

It dynamically generates a valid YAML configuration based on the class list, ensuring the YOLO training pipeline knows the correct number of classes and their names.

13

What is the purpose of background palettes in synthetic generation?

[Code](#)

```
BG_PALETTES = [
    [(25,22,38),(40,35,55),(55,50,70)], # night station
    [(50,45,35),(70,60,40),(90,80,55)], # day concrete
    [(20,30,45),(30,45,60),(45,60,75)], # blue-grey
    [(35,25,25),(55,40,35),(70,55,45)], # warm interior
]
```

Multiple color palettes simulate different lighting conditions (night, day, indoor, warm). The palette is cycled based on image index. This improves model robustness to varying environments.

14

What tools can be used to annotate custom images?

Theory

- **LabelImg** — Open-source desktop tool, supports YOLO format
- **Roboflow** — Web-based annotation with auto-labeling features
- **CVAT** — Computer Vision Annotation Tool (open-source, web-based)
- **Label Studio** — Multi-format annotation platform

After annotation, export in **YOLO format** and place images in `data/dataset/images/` and labels in `data/dataset/labels/`.

15

What is the difference between training and validation sets?

Theory

Set	Purpose	Used During
Training	Model learns patterns from this data	Training (weights updated)
Validation	Evaluate model performance, tune hyperparameters	After each epoch (no weight updates)
Test	Final evaluation on unseen data	After training is complete

Typical split: 80% train, 20% validation. The model never trains on validation data, ensuring unbiased performance metrics.

8 Training Pipeline

1 Explain the complete training workflow in `train.py` .

[Code](#)

1. Parse command-line arguments (yaml, model, epochs, batch, etc.)
2. Verify dataset YAML exists
3. Import and load base YOLO model: `model = YOLO('yolov8n.pt')`
4. Build training kwargs dict with all parameters
5. Start training: `results = model.train(**train_kwargs)`
6. Copy best weights: `runs/train/navigation_model/weights/best.pt` → `models/navigation_model_best.pt`
7. Run validation: `metrics = model.val()`
8. Print mAP@0.5 and mAP@0.5:0.95 scores

2

What are the key training parameters and their defaults?

Code

Parameter	Default	Description
<code>--yaml</code>	<code>config/dataset.yaml</code>	Dataset configuration path
<code>--model</code>	<code>yolo8n.pt</code>	Base model for transfer learning
<code>--epochs</code>	50	Number of training epochs
<code>--imgsz</code>	640	Input image size
<code>--batch</code>	16	Batch size
<code>--device</code>	auto	cpu / 0 (GPU) / 0,1 (multi-GPU)
<code>--workers</code>	4	DataLoader worker threads
<code>--resume</code>	False	Resume from last checkpoint
<code>--augment</code>	False	Enable extra augmentation

3

What is the role of `exist_ok=True` in model training?

Code

```
model.train(  
    ...  
    project='runs/train',  
    name='navigation_model',  
    exist_ok=True,      # Don't raise error if folder exists  
)
```

By default, YOLO creates a new folder for each run (navigation_model, navigation_model2, etc.). `exist_ok=True` allows overwriting the same folder, keeping the project clean and avoiding duplicate run directories.

4

What is the difference between `best.pt` and `last.pt`?

Theory

- **`best.pt`** — Weights from the epoch with the highest validation mAP. This is the optimal model.
- **`last.pt`** — Weights from the final epoch. May be overfitted if training continued too long.

The project copies `best.pt` to the models directory:

```
best_src = os.path.join(project, name, 'weights', 'best.pt')
shutil.copy(best_src, 'models/navigation_model_best.pt')
```

5

What is `model.val()` and what metrics does it return?

Code

```
metrics = model.val()
print(f"mAP@0.5 : {metrics.box.map50:.4f}")      # mAP at IoU=0.5
print(f"mAP@0.5:0.95: {metrics.box.map:.4f}")    # mAP averaged over IoU 0.5-0.95
```

`model.val()` runs the model on the validation set and computes detection metrics. Higher mAP values indicate better detection performance (1.0 = perfect).

6

How does the web-based training differ from CLI training?

Code

Feature	CLI (<code>train.py</code>)	Web (<code>app.py /api/train</code>)
Execution	Blocking (synchronous)	Background thread (async)
Progress	Console output	Poll <code>/api/train/status</code>
Stop	Ctrl+C	<code>/api/train/stop</code>
Validation	Runs automatically after training	Not auto-run
Resume	<code>--resume</code> flag	Not supported via API

7

What is overfitting and how can it be prevented?

Theory

Overfitting occurs when the model memorizes training data instead of learning general patterns. Signs: high training accuracy, low validation accuracy.

Prevention methods used in this project:

- Data augmentation (flip, rotate, scale, mosaic)
- Using pretrained weights (transfer learning)
- Early stopping (save best.pt based on validation mAP)
- Separate validation set to monitor generalization
- Sufficient training data (100-200 images minimum)

8

What is mosaic augmentation?

Theory

Mosaic augmentation combines 4 training images into a single composite image by:

1. Randomly selecting 4 images from the training set
2. Cropping and resizing each to fit a quadrant
3. Combining them into one image

Benefits: Exposes the model to more objects and context per batch, improves detection of small objects, and reduces the need for large batch sizes. Enabled with `mosaic=1.0` (100% probability).

9

What is the DataLoader workers parameter?

Code

```
p.add_argument('--workers', type=int, default=4)
```

`workers=4` means 4 parallel threads load and preprocess training images while the GPU trains on the current batch. This prevents the GPU from waiting for data (I/O bottleneck). More workers = faster data loading, but more CPU/RAM usage.

10

What does `--resume` do in training?

Code

```
p.add_argument('--resume', action='store_true')
# ...
train_kwargs['resume'] = args.resume
```

When `--resume` is set, training continues from the last saved checkpoint (`last.pt`) instead of starting fresh. This is useful when training is interrupted (power failure, manual stop) and you want to continue from where it left off without losing progress.

11

What is the role of GPU in training vs. CPU?

Theory

Aspect	CPU Training	GPU Training
Speed	Very slow (hours/days)	Fast (minutes/hours)
Parallelism	Limited cores (4-16)	Thousands of CUDA cores
Memory	System RAM	VRAM (4-24+ GB)
Cost	No extra hardware	NVIDIA GPU required

The project notes: "Training on CPU is slow; use GPU if available." Inference (detection) on CPU is acceptable for real-time with YOLOv8n.

```
# Training output directory structure:
runs/train/navigation_model/
├─ weights/
│   ├─ best.pt      # Best validation mAP
│   └─ last.pt      # Last epoch
├─ results.csv      # Metrics per epoch
├─ confusion_matrix.png
├─ F1_curve.png
├─ P_curve.png
├─ R_curve.png
└─ results.png

# Copy best model to accessible location:
shutil.copy('runs/train/navigation_model/weights/best.pt',
            'models/navigation_model_best.pt')
```

9 Configuration & Settings

1 Explain the Settings class in `config/settings.py` .

Code

```
class Settings:
    SECRET_KEY = os.environ.get('SECRET_KEY', 'nav-assist-secret-2024')
    DEFAULT_CONF = 0.40 # Confidence threshold
    DEFAULT_IoU = 0.45 # NMS IoU threshold
    DEFAULT_MODEL = 'yolov8n.pt'

    ZONES = {
        # Station zone boundaries (fractional)
        'entrance': (0.30, 0.00, 0.70, 0.20),
        'main_hall': (0.20, 0.20, 0.80, 0.60),
        ...
    }

    CLASS_COLORS = {
        # BGR colors per class
        'person': (219, 86, 26),
        'wheelchair_user': (0, 140, 255),
        ...
    }

    DEFAULT_COLOR = (100, 200, 50)
```

All configuration is centralized in one class, making it easy to modify thresholds, zones, and colors without changing multiple files.

2 How does the SECRET_KEY work?

[Code](#)

```
SECRET_KEY = os.environ.get('SECRET_KEY', 'nav-assist-secret-2024')
```

Flask's SECRET_KEY is used for:

- Signing session cookies cryptographically
- CSRF protection in forms
- Flash message security

`os.environ.get()` first checks for an environment variable. If not set, falls back to the default string. In production, this should be a random secret set via environment variable.

3 How are zone boundaries defined and what do the coordinates mean?

[Code](#)

```
'entrance': (0.30, 0.00, 0.70, 0.20)
#           x1    y1    x2    y2
```

Coordinates are **fractions (0.0 to 1.0)** of the frame dimensions:

- `x1=0.30` → Zone starts at 30% from left
- `y1=0.00` → Zone starts at top
- `x2=0.70` → Zone ends at 70% from left
- `y2=0.20` → Zone ends at 20% from top

For a 1280×720 frame: Entrance = pixels (384, 0) to (896, 144).

4 Why are colors stored in BGR format?

[Code](#)

OpenCV uses BGR color ordering (historical from early camera hardware). Since all visualization is done with OpenCV functions (`cv2.rectangle` , `cv2.putText` , etc.), storing colors in BGR avoids conversion overhead:

```
'person': (219, 86, 26) # B=219, G=86, R=26 → Orange in RGB
```

5 How would you modify the system to add a new station zone?

Viva

1. Add zone to `Settings.ZONES` in `config/settings.py` with coordinates
2. Add zone to `ZONE_DISPLAY_NAMES` in `navigator.py`
3. Add connections in `STATION_GRAPH` with instructions and arrows
4. Add zone color in `Visualizer._draw_zone_overlay()`
5. Optionally add to `DESTINATIONS` list in `demo.py`

6 What is the role of `setup.bat` and `setup.sh` ?

Code

They automate project setup on Windows (`setup.bat`) and Linux/Mac (`setup.sh`):

1. Create a Python virtual environment
2. Install dependencies from `requirements.txt`
3. Create dataset directory structure
4. Run `prepare_dataset.py`

Usage: `setup.bat` (Windows) or `bash setup.sh` (Linux/Mac)

7 What happens when `DEFAULT_CONF` is increased to 0.80?

Viva

Increasing confidence threshold to 0.80 means:

- **Fewer detections** — Only highly confident detections (80%+) are shown
- **Fewer false positives** — Less incorrect detections
- **More false negatives** — May miss real passengers detected with lower confidence
- **Better precision, worse recall**

For a navigation assistance system, a lower threshold (0.40) is preferred — it's better to detect a passenger with a possible false alarm than to miss someone who needs help.

8

Why are zone coordinates normalized (0 to 1) instead of pixel values?

Viva

Normalized coordinates make the system **resolution-independent**:

- Same zones work for 640×480, 1280×720, or 1920×1080 cameras
- No need to reconfigure when changing camera resolution
- Zone proportions are maintained across different frame sizes
- Conversion is simple: $\text{pixel_x} = \text{fraction_x} \times \text{frame_width}$

10 Python Concepts Used

1 How is threading used in the project?

Code

Threading is used for 4 key background operations:

1. **Training:** `threading.Thread(target=run_training, daemon=True)`
 2. **Audio playback:** `threading.Thread(target=self.speak, daemon=True)`
 3. **Synthetic dataset generation:** Background thread in `/api/dataset/synthetic`
 4. **Roboflow download:** Background thread in `/api/dataset/roboflow`
- `threading.Lock()` is used for camera access synchronization and TTS engine thread safety.

2 What is a Python decorator? How is `@app.route()` used?

Theory

A **decorator** is a function that modifies another function's behavior. `@app.route()` is Flask's decorator that maps URLs to Python functions:

```
@app.route('/detect', methods=['GET'])    # URL: /detect
def detect_page():                       # View function
    return render_template('detect.html')

@app.route('/api/train', methods=['POST']) # POST to /api/train
def start_training():
    # Handle training request
```

It registers the function as a handler for the specified URL and HTTP method.

3

What is `argparse` and how is it used?

Code

```
p = argparse.ArgumentParser(description='...')
p.add_argument('--epochs', type=int, default=50)
p.add_argument('--save', action='store_true') # Boolean flag
p.add_argument('--mode', choices=['image', 'video', 'live'])
args = p.parse_args()

print(args.epochs) # 50 or user-specified value
```

`argparse` handles command-line argument parsing with type validation, defaults, choices, and auto-generated help messages.

4

What is a generator function? How is `generate_frames()` one?

Code

A **generator** is a function that uses `yield` instead of `return`. It produces values lazily (one at a time) without loading everything into memory:

```
def generate_frames():
    while camera_active:
        ret, frame = cap.read()
        # ... process frame ...
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' +
              buffer.tobytes() + b'\r\n')
```

Each `yield` sends one JPEG frame. The function pauses until the next frame is requested by the client. This is memory-efficient for streaming thousands of frames.

5 What is `os.path.join()` and why is it used?

[Code](#)

```
path = os.path.join('static', 'uploads', fname)
# Windows: 'static\\uploads\\filename.jpg'
# Linux:   'static/uploads/filename.jpg'
```

`os.path.join()` creates file paths using the correct separator for the operating system. This makes the code **cross-platform compatible** — it works on Windows (backslash) and Linux/Mac (forward slash).

6 Explain context managers (`with` statement) used in the project.

[Code](#)

```
# Thread lock
with camera_lock:                # Acquires lock, auto-releases on exit
    camera_active = True

# File operations
with open(lbl_path, 'w') as f:    # Opens file, auto-closes on exit
    f.write(label_data)

# Zip extraction
with zipfile.ZipFile(zpath, 'r') as z: # Opens zip, auto-closes
    z.extractall('data/dataset')
```

The `with` statement ensures resources are properly cleaned up (lock released, file closed, etc.) even if an exception occurs.

7

What is list comprehension and where is it used?

Code

```
# Filter model files
models = [f for f in os.listdir('models') if f.endswith('.pt')]

# Find train images in extracted zip
train_imgs = [f for f in image_files if 'train' in str(f)]
```

List comprehension creates lists concisely. It's equivalent to:

```
models = []
for f in os.listdir('models'):
    if f.endswith('.pt'):
        models.append(f)
```

8

What is the `global` keyword used for?

Code

```
def start_training():
    global training_status      # Access module-level variable
    training_status['running'] = True
    ...

def stop_camera():
    global camera_active        # Modify module-level variable
    camera_active = False
```

The `global` keyword tells Python to use the module-level variable instead of creating a local one. It's needed when **assigning** to a global variable inside a function. Reading globals doesn't require it.

9

What is f-string formatting?

Code

```
print(f"MAP@0.5: {metrics.box.map50:.4f}")      # 4 decimal places
print(f"Progress: {pct:.0f}% frame {idx}/{total}") # No decimals
fname = f"{ts}_{fname}"                        # String concatenation
text = f'{label} {int(conf * 100)}%'           # Expression
```

f-strings (formatted string literals) embed Python expressions inside strings using `{}`. They support format specifiers like `:.4f` (4 decimal float), `:.0f` (no decimals), `:>5` (right-aligned, width 5).

10

What is `shutil` used for?

Code

```
import shutil

shutil.copy(best_src, dst)          # Copy single file
shutil.copy2(img_path, dst_img)     # Copy with metadata (timestamps)
shutil.rmtree(extract_dir)          # Delete entire directory tree
```

`shutil` provides high-level file operations. `copy2()` preserves file metadata. `rmtree()` recursively deletes directories (like `rm -rf`).

11 What is exception handling and how is it used?

[Code](#)

```
try:
    from ultralytics import YOLO
    self.model = YOLO(model_path)
    return True, f"Model loaded: {model_path}"
except Exception as e:
    self.model = None
    return False, f"Failed to load model: {e}"
finally:
    training_status['running'] = False # Always runs
```

`try/except/finally` blocks handle errors gracefully. `except Exception as e` catches all exceptions and stores the error message. `finally` runs whether or not an exception occurred — used to clean up resources.

12 What is the `Path` class from `pathlib` ?

[Code](#)

```
from pathlib import Path

DATASET_ROOT = Path('data/dataset')
img_dir = DATASET_ROOT / 'images' / 'train' # Path joining with /
img_dir.mkdir(parents=True, exist_ok=True) # Create directories
list(img_dir.glob('*.*jpg')) # Find files
img.stem # Filename without extension
```

`Path` provides an object-oriented interface for filesystem paths. It's more readable than `os.path` and supports operator overloading (`/` for path joining).

11 Code-Level Viva Questions

1 What will happen if `detector.detect()` is called before `detector.load()` ? Viva

It will return `None` because `self.model` is `None` :

```
def detect(self, frame):  
    if not self.is_loaded(): # self.model is None → False  
        return None        # Returns None safely
```

Downstream code handles this: `format_results(None)` returns `[]`, and the Visualizer skips drawing if results is None.

2 Why is `verbose=False` used in `model.predict()` ? Viva

`verbose=False` suppresses console output for each prediction. Since `predict()` is called on every frame (30+ times per second), verbose logging would:

- Flood the console with output
- Slow down processing due to I/O overhead
- Make useful log messages hard to read

3 What does `results[0]` mean in the detect method? Viva

```
results = self.model.predict(frame, ...)  
return results[0] if results else None
```

`model.predict()` returns a **list** of Results objects — one per input image. Since we pass a single frame, we take `results[0]` (the first and only result). The `if results` check handles the edge case where an empty list is returned.

4 How does `set_conf()` prevent invalid values?

Viva

```
def set_conf(self, conf: float):  
    self.conf = max(0.01, min(0.99, conf))
```

It uses **clamping**: `min(0.99, conf)` caps the maximum at 0.99, and `max(0.01, ...)` sets the minimum to 0.01. This ensures the confidence threshold stays in a valid range (0.01 to 0.99), preventing edge cases like 0 (everything detected) or 1 (nothing detected).

5 What is the purpose of `frame.copy()` in the Visualizer?

Viva

```
def draw(self, frame, results, navigator, ...):  
    out = frame.copy()    # Create a copy to draw on
```

NumPy arrays are passed by reference. Without `.copy()`, drawing operations would modify the original frame, which could affect other processing (e.g., the raw frame is needed for encoding in streaming). The copy ensures the original frame remains unmodified.

6 Explain the line: `x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())`

Viva

- `box.xyxy[0]` — Gets the first (only) bounding box as a PyTorch tensor
- `.tolist()` — Converts tensor to Python list: `[123.45, 67.89, 456.12, 345.67]`
- `map(int, ...)` — Converts each float to integer: `[123, 67, 456, 345]`
- `x1, y1, x2, y2 = ...` — Tuple unpacking into 4 variables

This converts YOLO's float tensor coordinates to integer pixel values for OpenCV drawing functions.

7

What is the `*'mp4v'` syntax in `VideoWriter_fourcc` ?

Viva

```
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
# Equivalent to:
fourcc = cv2.VideoWriter_fourcc('m', 'p', '4', 'v')
```

The `*` operator **unpacks** the string into individual characters. `VideoWriter_fourcc` expects 4 separate character arguments, so `*'mp4v'` passes `'m'`, `'p'`, `'4'`, `'v'` as separate arguments.

8

Why does `generate_frames()` use `try/finally` ?

Viva

```
try:
    while camera_active:
        ret, frame = cap.read()
        # ... process and yield frames ...
finally:
    cap.release()                # Always release camera
    with camera_lock:
        camera_active = False    # Always reset state
```

`finally` ensures the camera is released and state is reset even if an exception occurs (camera disconnects, encoding fails, client disconnects). Without it, the camera could remain locked and unavailable.

9

What is `rsplit('.', 1)` and why is `1` passed?

Viva

```
filename.rsplit('.', 1)[1].lower()
# "photo.backup.jpg" → ['photo.backup', 'jpg'] → 'jpg'
```

`rsplit('.', 1)` splits from the **right** with max 1 split. This correctly handles filenames with multiple dots. `split('.', 1)` from the left would give `['photo', 'backup.jpg']` — wrong extension.

10 How does `int(time.time()) % 3 == 0` work for periodic audio?

Viva

`time.time()` returns Unix timestamp (e.g., 1700000000.123). `int()` truncates to seconds. `% 3 == 0` is true every 3 seconds (when timestamp is divisible by 3).

This is a simple timer mechanism that triggers audio every ~3 seconds without needing a separate timer thread. Combined with the 4-second debounce, it prevents rapid-fire announcements.

11 What is `action='store_true'` in argparse?

Viva

```
p.add_argument('--save',      action='store_true')
p.add_argument('--no-audio',  action='store_true')
```

`action='store_true'` creates a boolean flag. If the flag is present, the value is `True`; if absent, it's `False`. No value argument is needed:

```
python detect.py --save      # args.save = True
python detect.py             # args.save = False
```

12 What does `cv2.addWeighted(overlay, 0.12, img, 0.88, 0, img)` do?

Viva

Formula: `output =  $\alpha$  \times src1 + \beta \times src2 + \gamma`

- `overlay` — Image with filled colored rectangles (zones)
- `0.12` (α) — 12% weight for overlay (subtle transparency)
- `img` — Original frame
- `0.88` (β) — 88% weight for original (dominant)
- `0` (γ) — No brightness offset
- Last `img` — Output destination (modifies in-place)

Result: Zones appear as 12% transparent colored overlays on the original frame.

13

What is the `Passenger` class in `demo.py`'s synthetic mode?

Viva

```
class Passenger:
    def __init__(self, cls, x, y):
        self.cls = cls
        self.x = float(x)
        self.y = float(y)
        self.vx = random.uniform(-1.2, 1.2) # velocity x
        self.vy = random.uniform(-0.5, 0.5) # velocity y
        self.conf = random.uniform(0.65, 0.97) # simulated confidence

    def update(self):
        self.x += self.vx
        self.y += self.vy
        if self.x < 30 or self.x > W-80: self.vx *= -1 # bounce
        if self.y < 50 or self.y > H-100: self.vy *= -1
```

It simulates moving passengers with random velocities. They bounce off screen edges, creating an animated demo without real YOLO detection.

14

How does exponential moving average FPS smoothing work in `demo.py`?

Viva

```
fps = 0.9 * fps + 0.1 * (1.0 / max(now - prev_time, 0.001))
```

This is an **exponential moving average (EMA)**:

- 90% weight on previous FPS value (smoothing)
- 10% weight on current frame's FPS (new measurement)
- Prevents jumpy FPS display from frame-to-frame variations
- `max(..., 0.001)` prevents division by zero

15

What is the `cv2.getTextSize()` used for?

Viva

```
(tw, th), _ = cv2.getTextSize(text, FONT, 0.55, 1)
ty = max(y1 - 8, th + 4)
cv2.rectangle(img, (x1, ty-th-4), (x1+tw+8, ty+2), color, -1) # Background
cv2.putText(img, text, (x1+4, ty-2), FONT, 0.55, (255,255,255), 1)
```

`getTextSize()` returns the width and height of a text string at a given font and scale. This is used to draw a precisely-sized colored background rectangle behind label text, ensuring the text is always readable regardless of the underlying image content.

16

What does `os.makedirs('models', exist_ok=True)` do?

Viva

`os.makedirs()` creates a directory and all intermediate parent directories. `exist_ok=True` prevents an error if the directory already exists. Without it, creating an existing directory raises `FileExistsError`.

```
os.makedirs('models', exist_ok=True) # Create models/ if not exists
os.makedirs(UPLOAD_FOLDER, exist_ok=True) # Create static/uploads/
```

17

How does the system handle concurrent training requests?

Viva

```
if training_status['running']:
    return jsonify({'success': False, 'message': 'Training already running'})
```

The system checks the `training_status['running']` flag before starting new training. If training is already in progress, it rejects the request with an error message. This prevents multiple simultaneous training sessions that would compete for GPU/CPU resources.

18

What is the `replace('\\', '/')` used for in result paths?

Viva

```
'result_image': '/' + out_path.replace('\\', '/')
```

On Windows, `os.path.join()` uses backslashes (`static\uploads\file.jpg`). Web URLs use forward slashes. This conversion ensures the file path works as a valid URL in the browser. The leading `/` makes it an absolute URL path.

19

Why is `frame.shape[:2]` used instead of `frame.shape` ?

Viva

```
h, w = frame.shape[:2]
```

OpenCV frames are NumPy arrays with shape `(height, width, channels)` — e.g., `(720, 1280, 3)` . Using `[:2]` takes only the first 2 values (height, width), ignoring the 3rd (color channels). This works for both color (3 channels) and grayscale (no channel dimension) images.

20

What is the significance of `quality=88` in synthetic image saving?

Viva

```
img.save(img_dir / f"{name}.jpg", quality=88)
```

JPEG quality ranges from 1 (worst) to 100 (best). Quality 88 is a good balance between:

- **File size** — Lower than quality 95 (default), saving disk space for large datasets
- **Visual quality** — Minimal visible compression artifacts at 88
- **Training benefit** — Slight compression actually helps as augmentation (the model learns to handle compressed images)

12 Real-World & Application Questions

1 What are the real-world applications of this system?

Theory

- **Railway stations:** Guide differently-abled passengers to platforms, restrooms, exits
- **Airports:** Navigate visually impaired travelers to gates
- **Hospitals:** Direct patients in wheelchairs to departments
- **Shopping malls:** Assist elderly shoppers with navigation
- **Smart cities:** Integrate with IoT for automated assistance
- **Disaster management:** Guide evacuees to exits

2 What are the limitations of this system?

Theory

- **Fixed station layout:** Zone boundaries are hardcoded and frame-dependent
- **Single camera:** Cannot track passengers across multiple cameras
- **Synthetic training data:** Real-world performance requires real annotated images
- **Lighting sensitivity:** YOLO performance degrades in poor lighting
- **Privacy concerns:** Camera-based detection raises surveillance issues
- **Internet dependency:** gTTS fallback requires internet
- **No obstacle detection:** Doesn't detect stairs, gaps, or physical barriers

3

How could this system be improved?

Theory

- **Multi-camera tracking:** Track passengers across multiple cameras using Re-ID
- **Dynamic zone mapping:** Use SLAM or marker-based localization instead of fixed zones
- **Mobile app:** Smartphone-based version with haptic feedback
- **Obstacle detection:** Detect stairs, escalators, and barriers
- **Indoor positioning:** Combine with BLE beacons for precise localization
- **Multi-language TTS:** Support regional languages
- **Edge deployment:** Run on Jetson Nano or Raspberry Pi for cost efficiency
- **Real dataset:** Collect and annotate real station images

4

How does this project address accessibility?

Theory

- **Visual guidance:** On-screen arrows and zone overlays for sighted staff
- **Audio guidance:** TTS announcements for visually impaired passengers
- **Multi-class detection:** Different assistance for wheelchair users vs. blind persons
- **Route optimization:** BFS finds shortest path, reducing walking distance
- **Elevator awareness:** Routes wheelchair users through elevator zone
- **Real-time processing:** Immediate feedback for time-sensitive navigation

5

What ethical considerations apply to this system?

Theory

- **Privacy:** Camera surveillance of public spaces — need consent/notice
- **Bias:** Model may perform differently across demographics if training data is biased
- **Misclassification:** Incorrectly labeling someone as "elderly" or "disabled" could be offensive
- **Data storage:** Recorded footage must comply with data protection laws (GDPR, etc.)
- **Reliability:** System failures in safety-critical navigation could be dangerous
- **Consent:** Passengers should be informed about AI-based monitoring

6

How does this project compare to existing navigation systems?

Theory

Feature	This Project	Google Maps / NaviLens
Detection	Camera-based AI (YOLO)	GPS / QR codes
Indoor Use	Yes (camera-based)	Limited (no GPS indoors)
Accessibility Focus	Primary goal (6 classes)	General navigation
Audio Guide	Yes (pyttsx3/gTTS)	Yes
Infrastructure	Camera + server	Smartphone only
Cost	Low (open-source)	Free (commercial)

7

What hardware is needed to deploy this system?

Theory

- **Camera:** USB webcam, IP camera, or CCTV with RTSP stream
- **Server:** PC with Python 3.8+ (CPU works; GPU recommended for training)
- **Speaker:** For audio announcements at station
- **Network:** LAN for web UI access; optional internet for gTTS
- **Minimum specs:** 4GB RAM, modern CPU (i5+), 10GB disk space
- **Recommended:** NVIDIA GPU (RTX 2060+) for training, 16GB RAM

8

How would you scale this system for a large railway station?

Theory

1. Deploy multiple cameras covering different areas
2. Run separate YOLO instances per camera feed
3. Use a central server to aggregate detections
4. Implement multi-camera tracking with Re-ID
5. Expand the zone graph for the actual station layout
6. Use message queues (Redis/RabbitMQ) for inter-service communication
7. Deploy on GPU server cluster for real-time multi-stream processing
8. Add a mobile app for individual passenger guidance

9

What is the SDG (Sustainable Development Goal) relevance of this project?

Theory

- **SDG 10 — Reduced Inequalities:** Provides equal access to transportation for differently-abled persons
- **SDG 11 — Sustainable Cities:** Makes public transport infrastructure accessible and inclusive
- **SDG 3 — Good Health:** Supports safe mobility for elderly and disabled passengers
- **SDG 9 — Industry & Innovation:** Uses AI technology for social infrastructure improvement

10

Summarize the complete system in 5 key points for a presentation.

Theory

1. **Problem:** Differently-abled passengers face difficulties navigating complex railway stations without assistance.
2. **Solution:** AI-powered system using YOLOv8 object detection to identify 6 types of passengers (wheelchair users, blind persons, elderly, etc.) from camera feeds.
3. **Navigation:** BFS graph algorithm routes passengers through 9 station zones with step-by-step instructions and directional arrows.
4. **Output:** Real-time visual annotations (bounding boxes, zone overlays, arrows) and audio guidance (pyttsx3/gTTS) for immediate, accessible assistance.
5. **Interface:** Full Flask web application with dashboard, training UI, detection modes (image/video/live), and interactive route planner — all running locally for privacy.

Navigation Assistance System for Differently-Abled Passengers

Total Questions: 167 | Topics: 12 Sections

Technologies: Python, YOLOv8, Flask, OpenCV, BFS, pyttsx3, gTTS, PIL, PyTorch

Generated for exam preparation • Print via browser: Ctrl+P → Save as PDF